

Laporan Simulasi Computer Vision: Implementasi Pengenalan Wajah dan Deteksi Gestur Tangan

Teknologi Berkembang (C)



Disusun oleh:

- | | |
|-----------------------------|------------|
| 1. Davin Jonathan Tanus | 5026231131 |
| 2. Adifa Fajri Sampurno | 5026231178 |
| 3. Daniel Sandi Bratanata A | 5026231216 |

Dosen Pengampu:

Izzat Aulia Akbar, S.Kom., M.Eng., Ph.D. - 1991202011006

**DEPARTEMEN SISTEM INFORMASI
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
TAHUN AJARAN 2025/2026**

DAFTAR ISI

TAHUN AJARAN 2025/2026	0
DAFTAR ISI	1
BAB 1	
PENDAHULUAN	2
1.1 Latar Belakang	2
1.2 Tujuan	3
1.3 Manfaat	3
BAB 2	
LANDASAN TEORI	4
2.1 Computer Vision	4
2.2 OpenCV	4
2.3 MediaPipe	5
2.4 Haar Cascade Classifier	6
2.5. LBPH Face Recognizer	7
2.6 MobileNetV2	8
BAB 3	
SIMULASI Pengerjaan	9
3.1. Simulasi Deteksi Wajah	9
3.2. Simulasi Deteksi Rangka Tangan	10
3.3. Simulasi dan Improvisasi Pengenalan Wajah	12
BAB 4	
KESIMPULAN	23
4.1. Kesimpulan	23
DAFTAR PUSTAKA	24

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi kecerdasan buatan (Artificial Intelligence) telah membawa perubahan signifikan dalam berbagai aspek kehidupan manusia. Salah satu cabang AI yang berkembang pesat dan memiliki dampak luas adalah Computer Vision. Computer Vision merupakan bidang ilmu yang memungkinkan komputer atau sistem untuk "melihat", menginterpretasi, dan memahami informasi visual dari dunia nyata, seperti gambar dan video, layaknya manusia. Tujuan utama dari teknologi ini adalah untuk mereplikasi hingga melampaui kemampuan visual manusia dalam mengenali dan memproses objek visual secara otomatis. Kemampuannya untuk mengekstrak, menganalisis, dan memahami data visual telah membuka peluang inovasi di berbagai sektor industri.

Penerapan Computer Vision dapat ditemukan dalam berbagai aplikasi praktis, mulai dari bidang keamanan yang menggunakan deteksi anomali pada rekaman CCTV, industri otomotif dengan pengembangan kendaraan otonom, hingga sektor kesehatan untuk analisis citra medis seperti Sinar-X dan MRI. Salah satu implementasi yang paling dikenal oleh masyarakat luas adalah teknologi pengenalan wajah (Face Recognition). Teknologi ini secara spesifik berfungsi untuk mengidentifikasi atau memverifikasi identitas seseorang berdasarkan fitur-fitur wajah mereka dari sebuah gambar digital atau cuplikan video. Penggunaannya sangat masif, mulai dari sistem keamanan pada gawai pribadi, sistem absensi di perkantoran, hingga pengawasan di area publik.

Seiring dengan meningkatnya kebutuhan akan sistem identifikasi yang akurat dan efisien, penelitian dan pengembangan dalam bidang Face Recognition menjadi sangat relevan. Laporan ini disusun untuk mendalami implementasi praktis dari teknologi Computer Vision melalui serangkaian simulasi yang telah dipaparkan dalam materi perkuliahan. Secara khusus, laporan ini akan berfokus pada simulasi Face Recognition dengan melakukan improvisasi sesuai dengan instruksi yang diberikan. Improvisasi tersebut mencakup pengembangan sistem agar mampu mendeteksi dan mengenali minimal tiga wajah orang yang berbeda, serta menerapkan upaya untuk meningkatkan akurasi dari hasil pengenalan wajah. Melalui pengerjaan tugas ini, diharapkan dapat diperoleh pemahaman yang lebih mendalam mengenai prinsip kerja, tantangan, serta potensi pengembangan teknologi Face Recognition di masa depan.

1.2 Tujuan

Laporan ini disusun dengan tujuan utama untuk melakukan eksplorasi praktis terhadap beberapa aplikasi fundamental dari teknologi *computer vision*. Secara lebih terperinci, tujuan dari laporan ini adalah sebagai berikut:

1. Melaksanakan serangkaian simulasi teknologi *computer vision* sesuai dengan materi yang disajikan, meliputi deteksi wajah dasar dan deteksi kerangka tangan (*hand skeleton*).
2. Mengimplementasikan dan melakukan improvisasi pada sistem pengenalan wajah (*face recognition*), dengan mengembangkan sistem yang mampu mengenali minimal tiga individu yang berbeda.
3. Melakukan upaya untuk meningkatkan tingkat akurasi pada sistem pengenalan wajah yang dikembangkan, melalui optimalisasi proses pengumpulan data dan penyesuaian parameter pada algoritma.

1.3 Manfaat

Penulisan laporan ini diharapkan dapat memberikan manfaat signifikan. Secara teoretis, laporan ini dapat menjadi referensi yang mendokumentasikan proses implementasi dan analisis kinerja dari beberapa algoritma dasar *computer vision*. Hasil dari simulasi dan improvisasi yang dilakukan dapat memberikan wawasan mengenai faktor-faktor yang memengaruhi akurasi dan keandalan sistem. Secara praktis, laporan ini menghasilkan serangkaian kode program yang fungsional dan dapat dijadikan dasar untuk pengembangan proyek-proyek *computer vision* yang lebih kompleks di masa depan. Selain itu, laporan ini memberikan pengalaman langsung dalam mengatasi tantangan teknis dalam membangun sebuah sistem pengenalan wajah, mulai dari persiapan dataset hingga pengujian model.

BAB 2

LANDASAN TEORI

2.1 Computer Vision

Computer Vision adalah cabang dari kecerdasan buatan (AI) yang melatih komputer untuk menginterpretasi dan memahami dunia visual. Dengan menggunakan gambar digital dari kamera dan video serta model pembelajaran mendalam, mesin dapat secara akurat mengidentifikasi dan mengklasifikasikan objek, lalu bereaksi terhadap apa yang "dilihat". Tujuan dari Computer Vision adalah untuk mengotomatisasi tugas-tugas yang dapat dilakukan oleh sistem visual manusia, bahkan melampauinya dalam beberapa aspek.

Dalam Computer Vision, kemajuan signifikan telah dicapai di berbagai bidang yang melibatkan klasifikasi gambar, segmentasi semantik, deteksi objek, dan rekonstruksi super-resolusi gambar seiring dengan pesatnya perkembangan *deep convolutional neural network* (CNN) (Zhao et al., 2024). CNN menjadi fondasi bagi mayoritas algoritma Computer Vision modern karena kemampuannya untuk melakukan pembelajaran fitur secara mandiri. Melalui proses pelatihan, model CNN dapat mengekstraksi fitur-fitur relevan dari data masukan mentah, mulai dari fitur tingkat rendah seperti tepi dan tekstur hingga fitur tingkat tinggi seperti bentuk dan objek utuh.

2.2 OpenCV

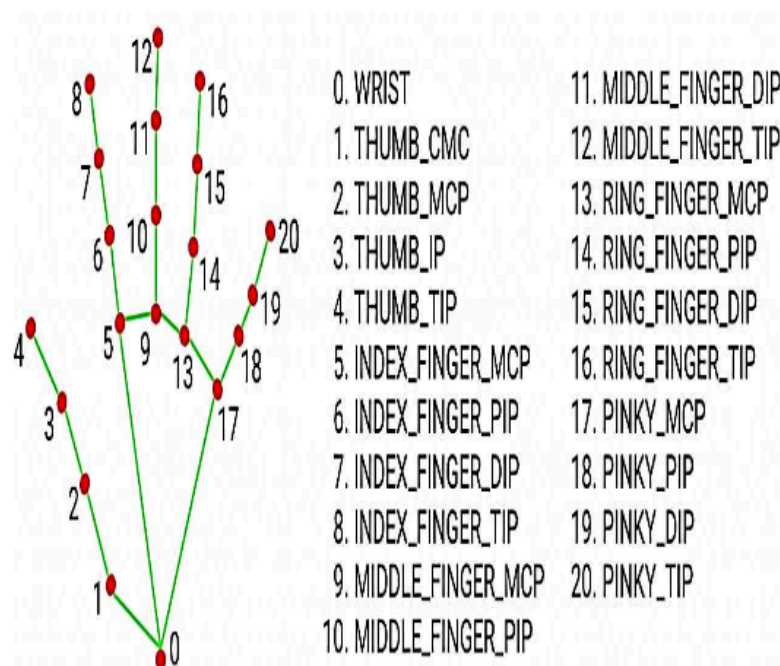
OpenCV (*Open Source Computer Vision Library*) adalah sebuah pustaka pemrograman sumber terbuka yang ditujukan untuk pemrosesan citra dan Computer Vision secara *real-time*. Pustaka ini menyediakan ribuan algoritma yang telah dioptimalkan, yang dapat digunakan untuk berbagai tugas seperti deteksi dan pengenalan wajah, identifikasi objek, pelacakan objek bergerak, ekstraksi model 3D, dan banyak lagi. OpenCV ditulis dalam C++ dan memiliki antarmuka untuk berbagai bahasa pemrograman populer seperti Python, Java, dan MATLAB, membuatnya sangat fleksibel untuk pengembangan aplikasi lintas platform.

Fungsi utama OpenCV dalam banyak aplikasi Computer Vision adalah untuk menangkap bingkai dari aliran video kamera secara *real-time* (Anand et al., 2024). Setelah bingkai gambar ditangkap, OpenCV menyediakan serangkaian fungsi untuk melakukan pra-pemrosesan, seperti konversi ruang warna (misalnya dari RGB ke Grayscale), pengubahan ukuran, dan normalisasi. Selain itu, OpenCV juga dapat digunakan untuk melokalisasi posisi dan batasan tangan atau wajah dalam setiap bingkai video dengan menerapkan teknik pemrosesan gambar, baik segmentasi warna kulit maupun deteksi kontur.

2.3 MediaPipe

MediaPipe adalah kerangka kerja yang dikembangkan oleh Google untuk membangun alur pemrosesan persepsi multimodal yang dapat diaplikasikan pada beragam jenis data, mulai dari video, audio, hingga data sensor lainnya. Kerangka kerja ini bersifat lintas platform dan sangat fleksibel, memungkinkan pengembang untuk merancang dan menerapkan solusi *Computer Vision* canggih dengan performa tinggi di berbagai perangkat. MediaPipe menyediakan solusi siap pakai untuk tugas-tugas kompleks seperti pelacakan tangan, deteksi wajah, dan estimasi pose tubuh.

Salah satu keunggulan utama MediaPipe adalah kemampuannya dalam mengestimasi pose tangan secara akurat di dalam setiap bingkai video (Anand et al., 2024). Dengan memanfaatkan model jaringan saraf yang telah dilatih secara ekstensif, teknologi ini mampu mengidentifikasi 21 titik kunci (*keypoints*) pada tangan. Titik-titik ini mencakup setiap sendi jari dan telapak tangan, menyediakan representasi kerangka tangan yang detail. Informasi detail inilah yang memungkinkan sistem untuk mengenali gestur tangan yang kompleks, seperti mengepalkan tangan atau menunjuk, yang kemudian dapat dipetakan menjadi perintah kontrol spesifik dalam sebuah aplikasi.



Gambar 2.1 Ilustrasi 21 *keypoints* pada tangan

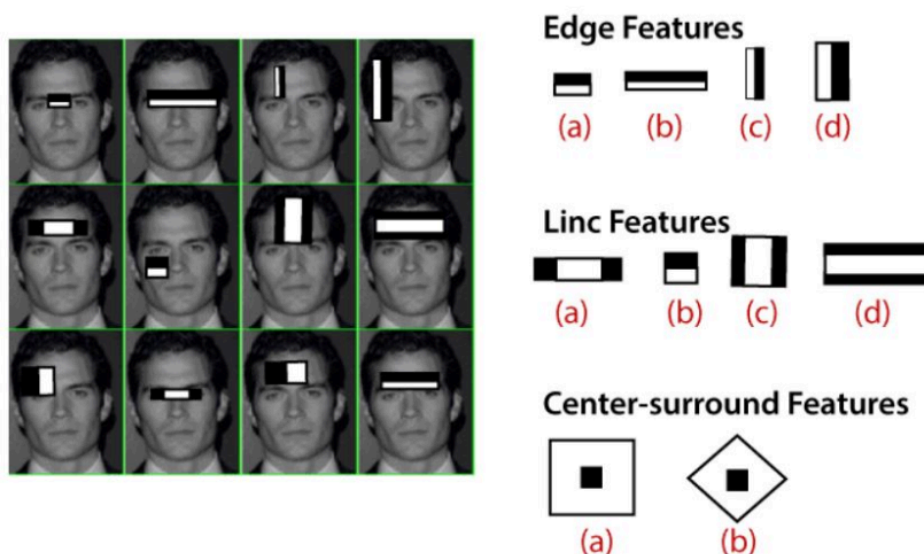
Di balik kemampuannya, MediaPipe untuk pelacakan tangan terdiri dari dua model utama yang bekerja secara sinergis: model deteksi telapak tangan (*palm detection*) dan model deteksi penanda tangan (*hand landmarks*). Pada awalnya, model deteksi telapak tangan yang lebih komputasional hanya dijalankan sekali untuk melokalisasi keberadaan tangan. Setelah itu, sistem

beralih menggunakan model penanda tangan yang lebih ringan untuk melacak 21 titik kunci pada bingkai-bingkai video berikutnya. Pendekatan dua tahap ini secara signifikan mengurangi beban komputasi, memastikan pelacakan tangan yang akurat dan efisien bahkan untuk aplikasi yang berjalan secara *real-time*.

2.4 Haar Cascade Classifier

Haar Cascade Classifier merupakan sebuah metode deteksi objek berbasis *machine learning* yang efektif dan cepat, dipopulerkan oleh Paul Viola dan Michael Jones. Algoritma ini dirancang untuk mendeteksi objek dalam gambar, dengan penerapan paling umum adalah untuk deteksi wajah. Metode ini bekerja dengan menggunakan fitur-fitur sederhana yang disebut *Haar-like features*. Fitur-fitur ini, yang terdiri dari kombinasi persegi panjang hitam dan putih, berfungsi untuk menghitung perbedaan jumlah intensitas piksel antara dua wilayah yang berdekatan untuk mengidentifikasi kontras yang menjadi ciri khas suatu objek.

Proses deteksi dilakukan dengan melatih sebuah model *classifier* menggunakan ribuan gambar positif (gambar yang mengandung wajah) dan negatif (gambar tanpa wajah). Dalam praktiknya, ketika mendeteksi wajah, metode Haar Cascade digunakan secara luas (Vijay et al., 2025). Selama proses deteksi, sebuah jendela pencarian digeser ke seluruh area gambar. Pada setiap posisi jendela, fitur-fitur Haar diekstraksi dan dievaluasi oleh serangkaian *classifier* lemah (*weak classifiers*). Jika sebuah wilayah berhasil melewati semua tahapan, wilayah tersebut ditandai sebagai deteksi wajah.



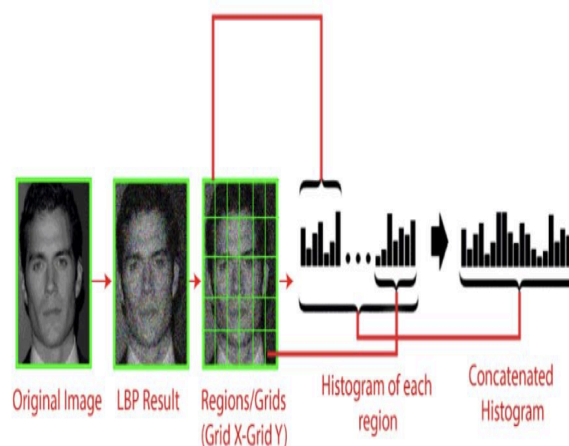
Gambar 2.2 LBPH feature extraction

Kunci dari efisiensi algoritma ini terletak pada konsep "kaskade" (*cascade*). Alih-alih menerapkan semua fitur pada setiap jendela, algoritma ini menyusun *classifier* lemah ke dalam beberapa tahapan. Tahapan awal terdiri dari *classifier* yang sangat sederhana dan cepat untuk segera menolak wilayah gambar yang jelas-jelas bukan wajah (misalnya, tembok polos). Hanya wilayah yang lolos dari tahapan awal yang akan diteruskan ke tahapan berikutnya yang lebih kompleks. Dengan pendekatan ini, sebagian besar wilayah negatif dapat dieliminasi dengan cepat, sehingga sumber daya komputasi dapat difokuskan hanya pada wilayah yang berpotensi mengandung wajah.

2.5. LBPH Face Recognizer

Local Binary Pattern Histograms (LBPH) adalah sebuah algoritma pengenalan wajah yang sederhana namun kuat, yang berfokus pada analisis tekstur lokal dari sebuah gambar. Keunggulan utama LBPH terletak pada ketahanannya terhadap perubahan kondisi pencahayaan, sebuah tantangan umum dalam tugas pengenalan wajah. Algoritma ini bekerja dengan cara membandingkan setiap piksel dengan piksel tetangganya dan menghasilkan representasi biner berdasarkan hasil perbandingan tersebut, sehingga menciptakan sebuah deskriptor tekstur yang unik.

LBPH merupakan teknik pengenalan wajah yang mengekstraksi fitur lokal dari sebuah gambar untuk kemudian digunakan dalam proses identifikasi (Chittibomma et al., 2024). Proses ini dimulai dengan membagi gambar wajah menjadi beberapa sel atau wilayah kecil. Untuk setiap sel, algoritma menghitung histogram dari pola-pola biner yang telah dihasilkan pada tahap sebelumnya. Kemudian, histogram dari semua sel digabungkan menjadi satu vektor fitur tunggal yang menjadi representasi numerik dari identitas wajah tersebut. Saat sebuah wajah baru perlu dikenali, proses yang sama diulang, dan vektor fiturnya dibandingkan dengan yang ada di basis data untuk menemukan kecocokan terdekat.



Gambar 2.3 Haar features learning dalam Fase training

Dibandingkan dengan metode berbasis *deep learning* yang memerlukan sumber daya komputasi besar dan dataset yang masif, LBPH menawarkan keunggulan dalam hal efisiensi. Algoritma ini secara komputasional lebih ringan dan dapat dilatih dengan jumlah data yang relatif sedikit, membuatnya ideal untuk aplikasi dengan keterbatasan perangkat keras. Ketahanannya terhadap pencahayaan berasal dari sifatnya yang hanya membandingkan intensitas relatif antar piksel tetangga, bukan nilai absolutnya. Selama kontras antara area terang dan gelap pada wajah (seperti mata, hidung, dan mulut) tetap terjaga, LBPH dapat menghasilkan pengenalan yang konsisten meskipun gambar diambil dalam kondisi pencahayaan yang berbeda-beda.

2.6 MobileNetV2

MobileNetV2 adalah arsitektur *Convolutional Neural Network* (CNN) yang sangat efisien, dirancang khusus oleh Google untuk aplikasi pada perangkat dengan sumber daya terbatas seperti ponsel dan sistem tertanam (*embedded systems*). Arsitektur ini merupakan penyempurnaan dari versi sebelumnya, MobileNetV1, dengan menawarkan akurasi yang lebih baik sambil tetap mempertahankan kompleksitas komputasi yang rendah. Inovasi utama dari MobileNetV2 adalah pengenalan modul lapisan baru yang disebut *inverted residual with a linear bottleneck*. Struktur ini secara cerdas memisahkan antara kapasitas representasi input/output (lapisan *bottleneck*) dan kemampuan transformasi non-linear dari lapisan perantara (*expansion layer*), sehingga memungkinkan pemrosesan fitur yang lebih kaya dengan jumlah operasi yang lebih sedikit.

Struktur blok fundamental pada MobileNetV2, yang dikenal sebagai *inverted residual*, bekerja secara terbalik dari blok residual tradisional seperti pada ResNet. Alih-alih mengompresi fitur terlebih dahulu, blok ini dimulai dengan lapisan konvolusi 1x1 untuk memperluas (*expand*) dimensi fitur, diikuti oleh konvolusi *depthwise* 3x3 untuk pemfilteran spasial, dan diakhiri dengan konvolusi 1x1 yang bersifat linier untuk memproyeksikan kembali fitur ke dimensi yang lebih rendah (*bottleneck*). Penggunaan aktivasi linier pada lapisan proyeksi akhir ini sangat krusial karena mencegah hilangnya informasi pada kanal berdimensi rendah, sebuah masalah yang sering terjadi jika fungsi non-linear seperti ReLU diterapkan. Kombinasi dari struktur residu terbalik dan *linear bottleneck* inilah yang menjadikan MobileNetV2 sangat efisien dan efektif untuk tugas-tugas *computer vision* di perangkat *mobile*.

BAB 3

SIMULASI Pengerjaan

3.1. Simulasi Deteksi Wajah

Langkah - langkah

1. Install opencv

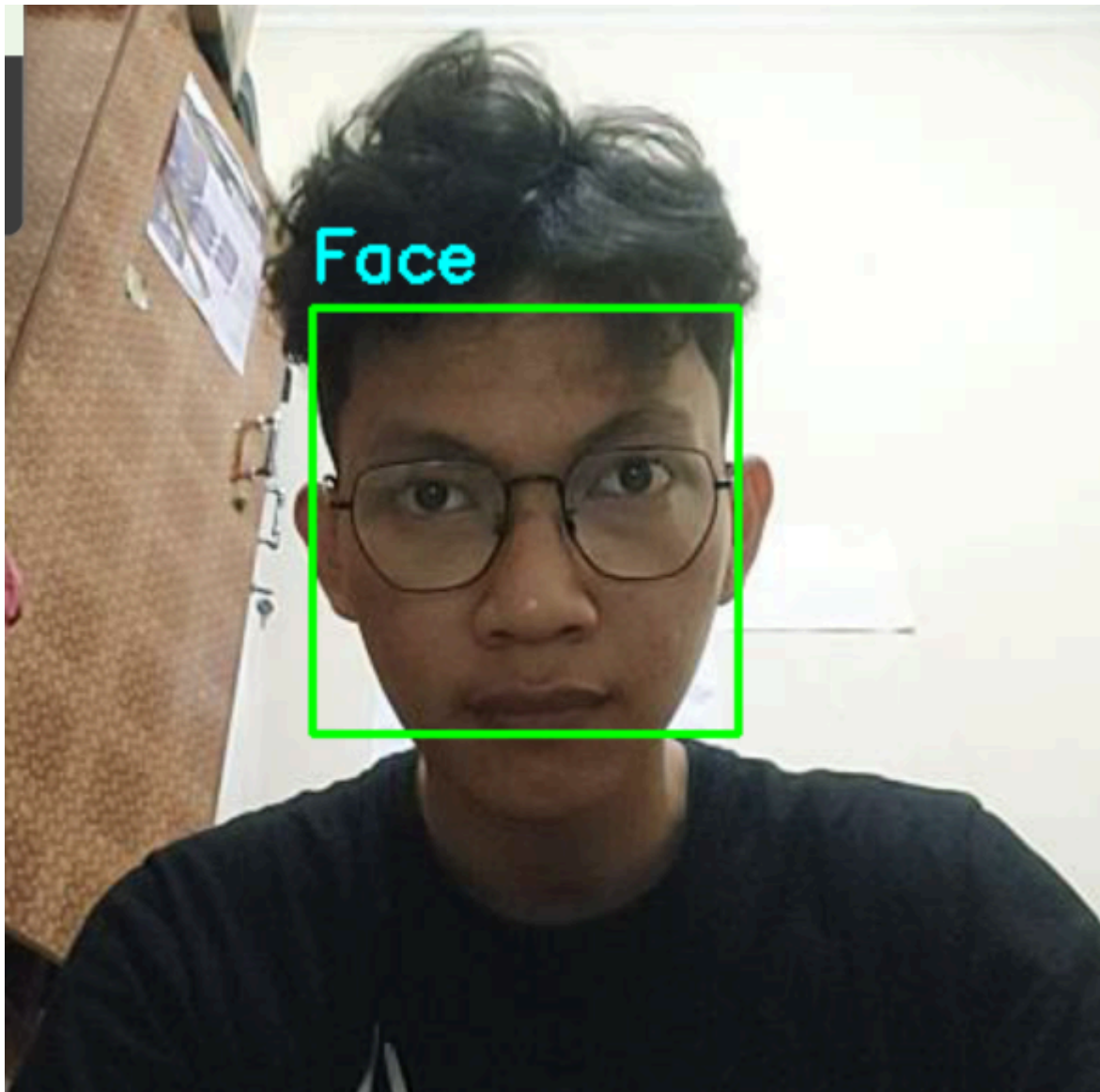
```
Anaconda Prompt

(base) C:\Users\micle>conda activate face_env

(face_env) C:\Users\micle>pip install opencv-contrib-python
Requirement already satisfied: opencv-contrib-python in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.17.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from opencv-contrib-python) (1.26.4)

(face_env) C:\Users\micle>
```

2. Testing



3. Source code

```
import cv2
import numpy as np

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
cap = cv2.VideoCapture(1)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5, minSize=(50, 50))

    for (x, y, w, h) in faces:
        x, y = max(0, x), max(0, y)
        face_crop = frame[y:y+h, x:x+w]
        if face_crop.size > 0:
            face_gray = cv2.cvtColor(face_crop, cv2.COLOR_BGR2GRAY)
            detected_face = cv2.resize(face_gray, (64, 64))
            cv2.imshow("Detected Face", detected_face)

            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
            cv2.putText(frame, "Face", (x, y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 0), 2)

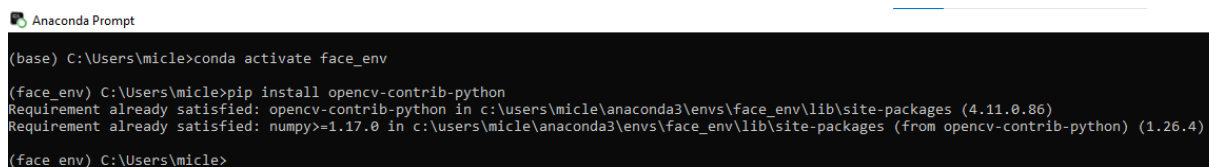
    cv2.imshow("Multi-face Detection (Haar Cascade)", frame)
    if cv2.waitKey(20) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

3.2. Simulasi Deteksi Rangka Tangan

Langkah - langkah

1. Install opencv



```
Anaconda Prompt

(base) C:\Users\micl>conda activate face_env

(face_env) C:\Users\micl>pip install opencv-contrib-python
Requirement already satisfied: opencv-contrib-python in c:\users\micl\anaconda3\envs\face_env\lib\site-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.17.0 in c:\users\micl\anaconda3\envs\face_env\lib\site-packages (from opencv-contrib-python) (1.26.4)

(face_env) C:\Users\micl>
```

2. Install mediapipe

```
(face_env) C:\Users\micle>pip install mediapipe
Requirement already satisfied: mediapipe in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (0.10.21)
Requirement already satisfied: absl-py in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (2.3.1)
Requirement already satisfied: attrs>=19.1.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (25.3.0)
Requirement already satisfied: flatbuffers>=2.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (25.9.23)
Requirement already satisfied: jax in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (0.4.30)
Requirement already satisfied: jaxlib in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (0.4.30)
Requirement already satisfied: matplotlib in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (3.9.4)
Requirement already satisfied: numpy<2 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (1.26.4)
Requirement already satisfied: opencv-contrib-python in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (4.11.0.86)
Requirement already satisfied: protobuf<5,>=4.25.3 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (4.25.8)
Requirement already satisfied: sounddevice>=0.4.4 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (0.5.2)
Requirement already satisfied: sentencepiece in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from mediapipe) (0.2.1)
Requirement already satisfied: CFFI>=1.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from sounddevice>=0.4.4->mediapipe) (2.0.0)
Requirement already satisfied: pycparser in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from CFFI>=1.0->sounddevice>=0.4.4->mediapipe) (2.23)
Requirement already satisfied: ml-dtypes>=0.2.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from jax->mediapipe) (0.5.3)
Requirement already satisfied: opt-einsum in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from jax->mediapipe) (3.4.0)
Requirement already satisfied: scipy>=1.9 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from jax->mediapipe) (1.13.1)
Requirement already satisfied: importlib-metadata>=4.6 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from jax->mediapipe) (8.7.0)
Requirement already satisfied: zipp>=3.20 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from importlib-metadata>=4.6->jax->mediapipe) (3.23.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (1.3.0)
Requirement already satisfied: cycler>=0.10 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (1.4.7)
Requirement already satisfied: packaging>=20.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (2.9.0.post0)
Requirement already satisfied: importlib-resources>=3.2.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from matplotlib->mediapipe) (6.5.2)
Requirement already satisfied: six>=1.5 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from python-dateutil>=2.7->matplotlib->mediapipe) (1.17.0)

(face_env) C:\Users\micle>
```

3. Source code

```
1 import cv2
2 import mediapipe as mp
3
4 cap = cv2.VideoCapture(0)
5 hands = mp.solutions.hands.Hands()
6 draws = mp.solutions.drawing_utils
7
8 while True:
9     ret, frame = cap.read()
10    if not ret:
11        break
12
13    frameRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
14    hand_obj = hands.process(frameRGB)
15
16    if hand_obj.multi_hand_landmarks:
17        for hand_landmarks in hand_obj.multi_hand_landmarks:
18            draws.draw_landmarks(frame, hand_landmarks, mp.solutions.hands.HAND_CONNECTIONS)
19
20    cv2.imshow("Hand Skeleton", frame)
21
22    if cv2.waitKey(1) & 0xFF == ord("q"):
23        break
24
25 cap.release()
26 cv2.destroyAllWindows()
27
```

4. Testing



3.3. Simulasi dan Improvisasi Pengenalan Wajah

Simulasi LBPH

1. Install opencv

```
Anaconda Prompt

(base) C:\Users\micle>conda activate face_env
(face_env) C:\Users\micle>pip install opencv-contrib-python
Requirement already satisfied: opencv-contrib-python in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.17.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from opencv-contrib-python) (1.26.4)
(face_env) C:\Users\micle>
```

2. Source Code For Making Dataset

```
import cv2
import os

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
video_path = "DANIEL.mp4"
dataset_path = "davindataset/daniel"
os.makedirs(dataset_path, exist_ok=True)
cap = cv2.VideoCapture(video_path)
count = 0
while True:
    ret, frame = cap.read()
    if not ret:
        print("Selesai ekstraksi frame.")
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        face_img = gray[y:y+h, x:x+w]
        file_name = f"{dataset_path}/daniel_{count:04d}.jpg"
        cv2.imwrite(file_name, face_img)
        count += 1
        print(f"Disimpan: {file_name}")

cap.release()
cv2.destroyAllWindows()

print(f"Total wajah tersimpan: {count}")
```


Dalam kasus ini kami menggunakan video wajah dari setiap anggota, yang kemudian dijadikan sampel untuk masing - masing dataset

```
▼ davindataset
  > adifa
  > daniel
  > davin
```

3. Code for training LBPH(Classification based histogram different pixel white and black)

```
import cv2
import numpy as np
import os

def checkDataset(directory="davindataset/"):
    return os.path.exists(directory) and len(os.listdir(directory)) != 0

def organizeDataset(root_path="davindataset/"):
    faces = []
    ids = []

    person_folders = [f for f in os.listdir(root_path) if
os.path.isdir(os.path.join(root_path, f))]
    print(f"Found {len(person_folders)} folders:", person_folders)

    for idx, folder in enumerate(person_folders):
        folder_path = os.path.join(root_path, folder)
        image_files = [os.path.join(folder_path, f) for f in
os.listdir(folder_path) if f.lower().endswith(('.jpg', '.png', '.jpeg'))]

        for img_path in image_files:
            img = cv2.imread(img_path)
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            faces_detected = faceCascade.detectMultiScale(gray,
scaleFactor=1.1, minNeighbors=5)

            for (x, y, w, h) in faces_detected:
                faces.append(gray[y:y+h, x:x+w])
                ids.append(idx)

    return faces, np.array(ids)

if not checkDataset():
    print("Dataset not found or empty.")
```

```

else:
    # inisialisasi recognizer dan face cascade
    recognizer = cv2.face.LBPHFaceRecognizer.create()
    faceCascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

    # train faces
    print("Training faces...")
    faces, ids = organizeDataset()
    if len(faces) == 0:
        print("No faces detected in dataset.")
    else:
        recognizer.train(faces, ids)
        recognizer.write("face-model.yml")
        print("Training finished!")
        print("Model saved as 'face-model.yml'")

```

4. Source code For Testing LBPH Model

```

import cv2
import os

model_path = "face-model.yml"
face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.read(model_path)

labels = {0: "Daniel", 1: "Davina", 2: "Adifa"}

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        print("Kamera tidak terdeteksi.")
        break

```



```

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.3, 5)

for (x, y, w, h) in faces:
    roi_gray = gray[y:y+h, x:x+w]
    id_, confidence = recognizer.predict(roi_gray)

    if confidence < 70:
        name = labels.get(id_, "Unknown")
        color = (0, 255, 0)
    else:
        name = "Unknown"
        color = (0, 0, 255)

    cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
    cv2.putText(frame, f'{name} ({round(confidence, 2)}%)', (x, y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

cv2.imshow("LBPH Face Recognition", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

5. Demo

Simulasi Improvisasi (Haarcascade head detection+Classifier mobilenetv2)

6. Install opencv

```
Anaconda Prompt

(base) C:\Users\micle>conda activate face_env

(face_env) C:\Users\micle>pip install opencv-contrib-python
Requirement already satisfied: opencv-contrib-python in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.17.0 in c:\users\micle\anaconda3\envs\face_env\lib\site-packages (from opencv-contrib-python) (1.26.4)

(face_env) C:\Users\micle>
```

7. Source Code For Making Dataset

```
import cv2
import os

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
video_path = "DANIEL.mp4"
dataset_path = "davindataset/daniel"
os.makedirs(dataset_path, exist_ok=True)
cap = cv2.VideoCapture(video_path)
count = 0
while True:
    ret, frame = cap.read()
    if not ret:
        print("Selesai ekstraksi frame.")
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    for (x, y, w, h) in faces:
        face_img = gray[y:y+h, x:x+w]
        file_name = f"{dataset_path}/daniel_{count:04d}.jpg"
        cv2.imwrite(file_name, face_img)
        count += 1
        print(f"Disimpan: {file_name}")

cap.release()
cv2.destroyAllWindows()

print(f"Total wajah tersimpan: {count}")
```

Dalam kasus ini kami menggunakan video wajah dari setiap anggota, yang kemudian dijadikan sampel untuk masing - masing dataset

```
▼ davindataset
  > adifa
  > daniel
  > davin
```

5. Code for Making Dataset

```
import cv2
import numpy as np
import os

def checkDataset(directory="davindataset/"):
    return os.path.exists(directory) and len(os.listdir(directory)) != 0

def organizeDataset(root_path="davindataset/"):
    faces = []
    ids = []

    person_folders = [f for f in os.listdir(root_path) if
os.path.isdir(os.path.join(root_path, f))]
    print(f"Found {len(person_folders)} folders:", person_folders)

    for idx, folder in enumerate(person_folders):
        folder_path = os.path.join(root_path, folder)
        image_files = [os.path.join(folder_path, f) for f in
os.listdir(folder_path) if f.lower().endswith(('.jpg', '.png', '.jpeg'))]

        for img_path in image_files:
            img = cv2.imread(img_path)
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            faces_detected = faceCascade.detectMultiScale(gray,
scaleFactor=1.1, minNeighbors=5)

            for (x, y, w, h) in faces_detected:
                faces.append(gray[y:y+h, x:x+w])
                ids.append(idx)

    return faces, np.array(ids)

if not checkDataset():
    print("Dataset not found or empty.")
```

```

else:
    # inisialisasi recognizer dan face cascade
    recognizer = cv2.face.LBPHFaceRecognizer.create()
    faceCascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

    # train faces
    print("Training faces...")
    faces, ids = organizeDataset()
    if len(faces) == 0:
        print("No faces detected in dataset.")
    else:
        recognizer.train(faces, ids)
        recognizer.write("face-model.yml")
        print("Training finished!")
        print("Model saved as 'face-model.yml'")

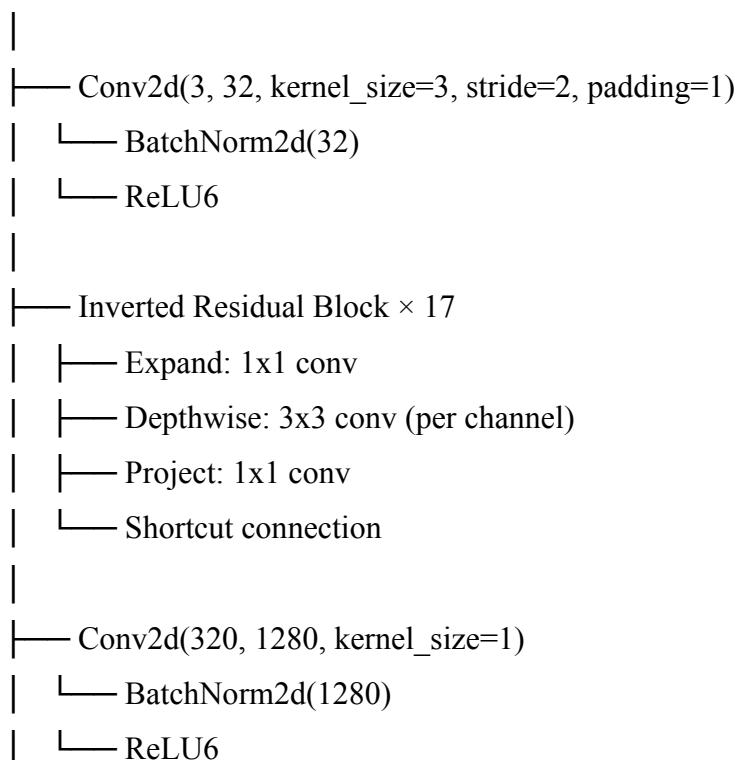
```

6. Source code For Training model mobilenetv2 to classify

https://drive.google.com/file/d/1KS_MBWotcR5my_l6HK2E55NSuG-vat2w/view?usp=sharing

We choose mobilenetv2 because of the model is not that complicated then VIT or DINO

Input Image (3 x 224 x 224)



```

|
|—— AdaptiveAvgPool2d(output_size=1)
|   ↓
|   Tensor shape: [batch, 1280, 1, 1]
|
|—— Flatten → [batch, 1280]
|
|—— Linear(1280 → num_classes)
|   ↓
Output logits (num_classes)

```

It just fine tune from pretrain model with >300 photo,parameter in this model around 13 million so its small enough and can be run on phone

7. Code For Testing our result

```

import cv2
import torch
import torch.nn as nn
import torchvision.transforms as transforms
import torchvision.models as models
import numpy as np
from PIL import Image

class FaceClassifier(nn.Module):
    def __init__(self, num_classes=5):
        super().__init__()
        base = models.mobilenet_v2(pretrained=True)
        base.classifier[1] = nn.Linear(base.last_channel, num_classes)
        self.model = base

    def forward(self, x):
        return self.model(x)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_classes = 3
model = FaceClassifier(num_classes=num_classes).to(device)
model.load_state_dict(torch.load("face_model_mobilenetv2 (1).pt",
                                map_location=device))

```

```

model.eval()

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

label_mapping = {
    "davin": 0,
    "daniel": 1,
    "adifa": 2,
}
reverse_labels = {v: k for k, v in label_mapping.items()}

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

cap = cv2.VideoCapture(1)
if not cap.isOpened():
    print("Kamera tidak terdeteksi.")
    exit()

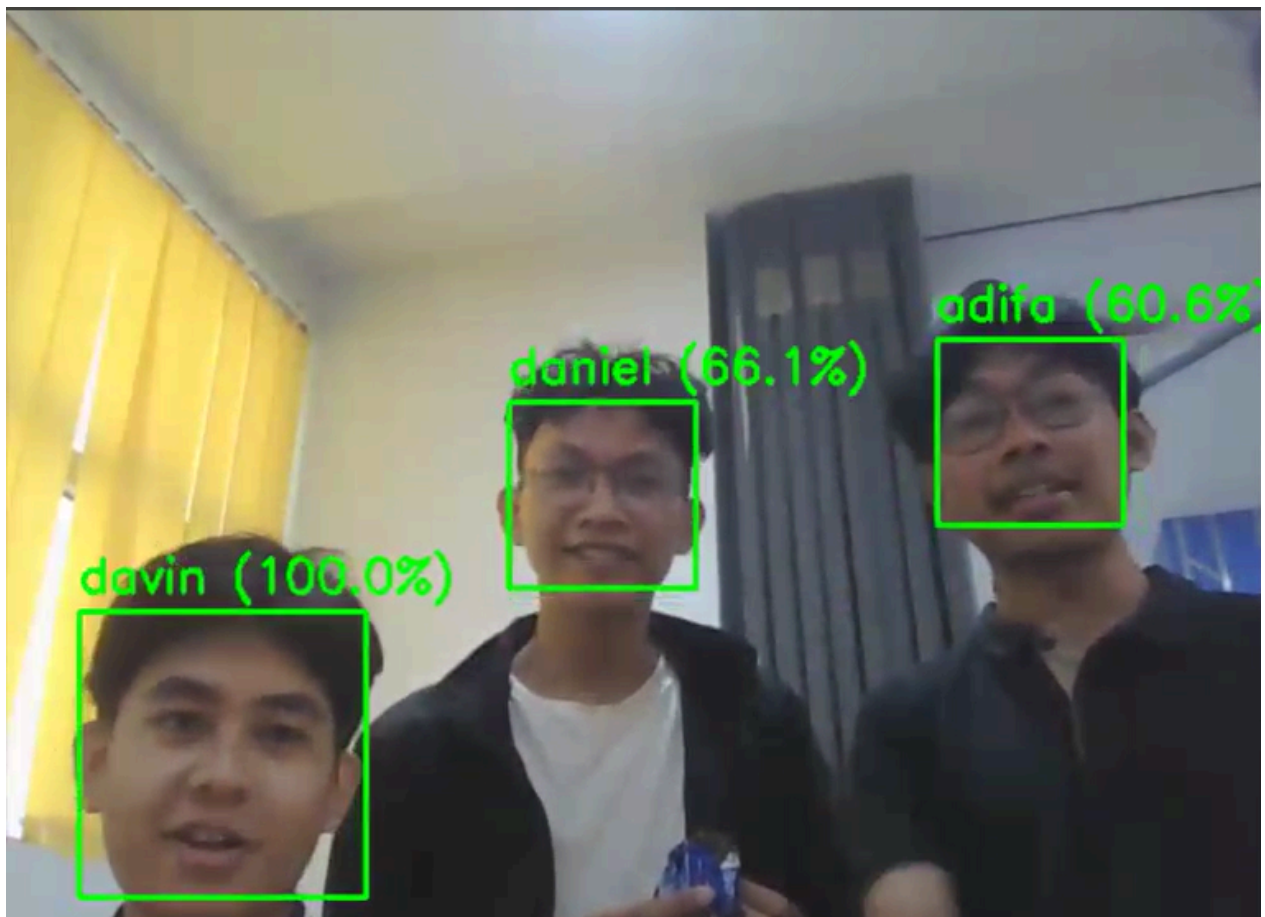
while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3,
minNeighbors=5)
    for (x, y, w, h) in faces:
        roi_color = frame[y:y+h, x:x+w]
        if roi_color.shape[0] < 50 or roi_color.shape[1] < 50:
            continue
        img_pil = Image.fromarray(cv2.cvtColor(roi_color,
cv2.COLOR_BGR2RGB))
        input_tensor = test_transform(img_pil).unsqueeze(0).to(device)
        with torch.no_grad():
            outputs = model(input_tensor)
            probs = torch.softmax(outputs, dim=1)
            conf, preds = torch.max(probs, 1)
            conf = conf.item()
            pred_label = preds.item()
            name = reverse_labels.get(pred_label, "N/A")

```

```
color = (0, 255, 0)
cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
cv2.putText(frame, f"{name} ({conf*100:.1f}%)", (x, y-10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
cv2.imshow("Face Recognition - MobileNetV2", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

8. Demo



BAB 4

KESIMPULAN

4.1. Kesimpulan

Berdasarkan hasil simulasi dan improvisasi yang telah dilakukan, dapat disimpulkan bahwa metode MobileNetV2 memberikan hasil yang jauh lebih baik dibandingkan dengan metode LBPH (Local Binary Pattern Histogram) dalam proses pengenalan wajah.

LBPH bekerja dengan prinsip sederhana, yaitu membedakan pola intensitas piksel dalam citra grayscale melalui perbandingan nilai hitam dan putih di sekitar piksel tertentu. Pendekatan ini cukup efisien dan mudah diimplementasikan, namun memiliki keterbatasan yang signifikan terhadap variasi pencahayaan, sudut wajah, serta ekspresi pengguna.

Sementara itu, MobileNetV2 merupakan model berbasis *deep learning* yang menggunakan *depthwise separable convolution* untuk mengekstraksi fitur secara lebih mendalam dan efisien. Model ini mampu memahami representasi visual yang lebih kompleks dan kontekstual dibandingkan LBPH. Dengan jumlah parameter sekitar 13 juta, MobileNetV2 tetap ringan namun memiliki kemampuan klasifikasi yang tinggi, bahkan dapat dijalankan pada perangkat dengan sumber daya terbatas seperti laptop standar atau ponsel pintar.

Hasil pengujian menunjukkan bahwa MobileNetV2 mampu mengenali wajah dengan akurasi dan konsistensi yang lebih tinggi dibandingkan LBPH. Hal ini membuktikan bahwa penggunaan *transfer learning* dengan model *pre-trained* seperti MobileNetV2 menjadi solusi modern yang efektif dalam tugas pengenalan wajah, menggantikan metode tradisional berbasis histogram. Selain itu, integrasi dengan Haar Cascade Classifier sebagai detektor awal wajah juga mempercepat proses identifikasi, karena model hanya perlu memproses area relevan pada setiap bingkai video.

Dengan demikian, dapat disimpulkan bahwa kombinasi Haar Cascade untuk deteksi dan MobileNetV2 untuk klasifikasi wajah merupakan pendekatan yang efisien, akurat, dan adaptif terhadap berbagai kondisi lingkungan, menjadikannya solusi yang lebih unggul dibandingkan algoritma konvensional seperti LBPH.

DAFTAR PUSTAKA

- Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024). A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, 57(1), 99. <https://doi.org/10.1007/s10462-024-10721-6>
- Anand, A., Pandey, A., Mehndiratta, S., Goyal, H., Kaushik, P., & Rathore, R. (2024). Smart AI based volume control system: Gesture recognition with OpenCV & Mediapipe integration. In *2024 International Conference on Smart Devices (ICSD)* (pp. 1–6). IEEE. <https://doi.org/10.1109/ICSD60021.2024.10751362>
- Vijay, K., Vimalan, M., Vishalini, E., Thilak Kumar, V., & Dhanush Ishwar, D. (2025). Face recognition based attendance system – A deep dive. In *Proceedings of the 3rd International Conference on Sustainable Computing and Data Communication Systems (ICSCDS-2025)* (pp. 819–825). IEEE. <https://doi.org/10.1109/ICSD65426.2025.11166971>
- Chittibomma, S. S., Surapaneni, R. K., & Maruboina, A. (2024). Facial recognition system for law enforcement: An integrated approach using Haar cascade classifier and LBPH algorithm. In *2024 International Conference on Advancements in Power, Communication and Intelligent Systems (APCI)* (pp. 1–7). IEEE. <https://doi.org/10.1109/APCI61480.2024.10616450>